SECTION 3 CHEAT SHEET

Fine-tune Laya in one call

Write the problem as one typed question and labelled rows, measure the published Laya, then `finetune` trains, calibrates and saves. On ten card queues accuracy went from 0.6775 to 0.9225; then check what it forgot.

ten queues, training messages each



1 THE PROBLEM AS DATA

One typed question, rows in Laya's format

A `choice` question whose criteria describe each queue, and one JSONL row per message: `state`, `questions`, `expected`. `finetune` trains on these rows and `laya.evals` scores them.

```
QUESTION = {
    "type": "choice",
    "instructions": "What does the customer need help with?",
    "criteria": INTENTS
}
rows.append({
    "state": message["text"],
    "questions": {"intent": QUESTION},
    "expected": {"intent": message["label_text"]}
})
train_table.to_json("runs/cards/train.jsonl",
                    orient="records", lines=True)
```

COURSE RUN Banking77 filtered to ten card intents: **1,120** training and **400** test messages.



0.6775

right on the 400 test messages

the two biggest mistakes, of 129

card_delivery_estimate — **29**
taken as card_arrival

compromised_card — **25**
taken as lost_or_stolen_card

2 MEASURE FIRST

The published Laya, before any training

`evaluate` sends every test row through the agent and scores it against `expected`: accuracy, ECE, Brier. Then read the wrong rows.

```
agent = laya.Agent(BASE, device="cuda")
test = Dataset.from_jsonl("runs/cards/test.jsonl")
before = evaluate(agent, test)
overall = pd.Series(before.overall)
cases = pd.DataFrame(before.cases)
wrong = cases[~cases["correct"]]
```

COURSE RUN ECE **0.1339**, Brier **0.1771**; the encoder is **ModernBERT-large**, **394,781,696** weights.

the 1,120 training rows inside finetune



1,008 trained 112 calibration

4 epochs, 64 updates, then one temperature



accuracy on the test rows, finetune's own table

3 FINETUNE

Train, calibrate, save: one call

`finetune` keeps 10% of the rows back, trains every weight with Laya's own loss, fits the temperatures on the held-back rows and saves a checkpoint. `laya.Agent` loads it like the Hub model.

```
from laya.train import finetune, TrainConfig
config = TrainConfig(eval_data="runs/cards/test.jsonl")
run = finetune("runs/cards/train.jsonl", BASE, OUT, config)
tuned = laya.Agent(OUT, device="cuda")
after = evaluate(tuned, test)
```

COURSE RUN ECE **0.1320** to **0.0536**; choice temperature **4.62**. By `evaluate` : **0.9225**, wrong **31**.

TrainConfig, the defaults finetune used

epochs	4
micro_batch	8
grad_accum	8
encoder_lr	2.5e-05
head_lr	0.0001
loss	'rlcd'
calib_frac	0.1

epoch loss



4 THE DEFAULTS

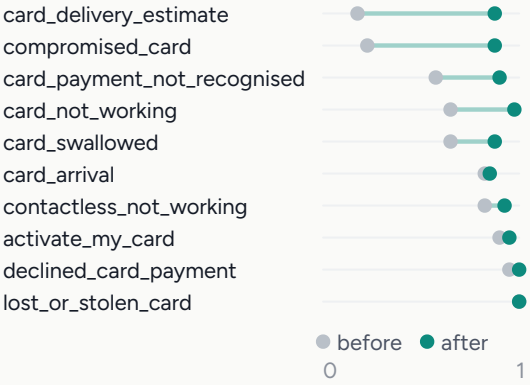
`TrainConfig`: what finetune runs with

One config holds the training choices: four epochs, the `rlcd` loss, one learning rate for the encoder and a larger one for the head, and a tenth of the rows kept back for calibration.

```
config = TrainConfig(eval_data="runs/cards/test.jsonl")
config
run["train_items"], run["calibration_items"], ...
run["epoch_loss"]
```

COURSE RUN **1,008** trained, **112** held back; the epoch loss rose again in epoch 3 (**0.688**), so judge by the test scores.

share right per queue, sorted by before



5 READ THE GAIN

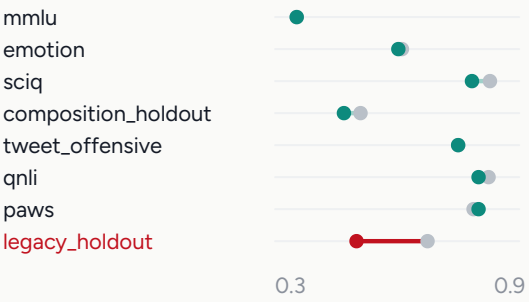
Every queue, before and after

Group the cases by the expected queue. The two weakest queues gained most, and no queue fell.

```
cases_after = pd.DataFrame(after.cases)
right_before = cases.groupby("expected")["correct"].mean()
right_after = cases_after.groupby("expected")[
    "correct"].mean()
```

COURSE RUN delivery estimate **0.175** to **0.875**, compromised **0.225** to **0.875**.

eight tasks with nothing to do with cards



6 FORGETTING

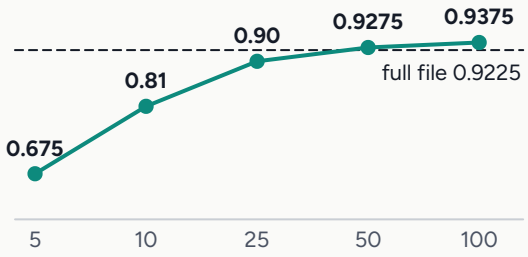
What one task can cost

Score both models on tasks the training never saw. `tags` name each task, and `slices["tag"]` reports every task on its own.

```
kev = pd.read_json(..., lines=True)
tags.append([record["_meta"]["source"]])
transfer = Dataset.from_jsonl("runs/cards/transfer.jsonl")
forget_after = evaluate(tuned, transfer)
by_task_after = pd.DataFrame.from_dict(
    forget_after.slices["tag"], orient="index")
```

COURSE RUN **764** rows from Kev's held-out file; legacy_holdout fell from **0.675** to **0.50**.

accuracy against messages per queue



7 HOW MANY LABELS

Accuracy against messages per queue

Each run starts again from the published Laya, on the first n messages of every queue, scored on the same 400 test rows.

```
for n in [5, 10, 25, 50, 100]:
    subset = train.groupby("intent").head(n)
    run_n = finetune(path, BASE, f"runs/laya-cards-{n}",
                     config)
    accuracy = run_n["train_report"]["after"]["accuracy"]
```

COURSE RUN 10 per queue already gave **0.81**; 100 per queue (**778** rows) **0.9375**.

Laya, then the fine-tuned Laya

Someone might have seen my card number …	
compromised_card	0.70
compromised_card	**0.93**
I got a text saying my card is on its way …	
card_arrival	0.99
card_arrival	**0.91**
The cash machine didn't give my card back!	
lost_or_stolen_card	0.52
card_swallowed	**0.91**
Why won't my card work when I tap it …	
contactless_not_working	0.68
card_not_working	**0.52**

8 TRY IT

New messages, both models

`predict` on both agents shows what fine-tuning changed, message by message.

```
for message in MESSAGES:
    old = agent.predict(message, {"intent": QUESTION})
    new = tuned.predict(message, {"intent": QUESTION})
pd.DataFrame(results)
```

COURSE RUN the cash machine message went from lost_or_stolen_card **0.52** to card_swallowed **0.91**; the tap at the shop got worse, contactless **0.68** to card_not_working **0.52**.

TERMS

fine-tune: continue training from a pretrained model on your data.

test set: the rows kept back to measure on.

calibration items: the 10% held back to fit the temperatures.

forgetting: a fine-tuned model losing skills it had.

WHAT EACH CALL HANDS BACK

`evaluate(agent, data)`
.overall, **.slices["tag"]**, **.cases**

`finetune(train, base, out, config)`
a dict: **train_items**, **temperature**, **train_report**

`TrainConfig()`
epochs **4**, loss **rlcd**, calib_frac **0.1**

`laya.Agent(OUT)`
the fine-tuned Laya, same API

WATCH OUT

- Forgetting: legacy_holdout **0.675** to **0.50**, sciq **0.828** to **0.784**.
- Five per queue changed nothing: still **0.675**.
- `evaluate` said **0.6775** before, `finetune`'s table **0.6750**.